



Happy Number

Company: JPMorgan

Year: 2023

Difficulty: Easy

Topic: HashSet, Math, Cycle Detection

Source: [LeetCode 202 – Happy Number](#)

Problem Description

Given a positive integer n , determine whether it is a **happy number**.

To determine whether a number is happy:

1. Take each digit of the number.
2. Square each digit.
3. Add all the squared digits.
4. Replace the original number with this sum.
5. Repeat the process.

If the process eventually reaches:

1

then the number is a **happy number**.

If the process enters a cycle that never reaches 1, then the number is **not happy**.

Return:

true

if n is happy, otherwise return:

false

Input Format

- The first line contains a single integer n — the positive integer that needs to be checked.

Output Format

- Print true if n is a happy number.
- Print false if n is not a happy number.

Constraints

- $1 \leq n \leq 2^{31} - 1$

Example 1

Input:

n = 19

Output:

true

Explanation

Starting with 19:

$$1^2 + 9^2 = 82$$

Then:

$$8^2 + 2^2 = 68$$

Then:

$$6^2 + 8^2 = 100$$

Then:

$$1^2 + 0^2 + 0^2 = 1$$

Since the process reaches 1, 19 is a happy number.

Example 2

Input:



n = 2



Output:

false

Explanation

Starting with 2:

$$2^2 = 4$$

Then:

$$4^2 = 16$$

Then:

$$1^2 + 6^2 = 37$$

Then:

$$3^2 + 7^2 = 58$$

The process eventually enters a cycle and never reaches 1.

Therefore:

false

Approach to Solve This Problem

The main challenge is that the process may **never reach 1**.

For example, some numbers repeatedly produce the same sequence of values. Therefore, we need to detect whether a number has appeared before.

We can use a **HashSet** to store all numbers that we have already seen.

For every number:

1. Calculate the sum of the squares of its digits.
2. If the result is 1, return true.

3. If the result has already appeared in the HashSet, a cycle exists, so return false.
4. Otherwise, add the result to the HashSet and continue.

Example

For:

19

The sequence is:

19 → 82 → 68 → 100 → 1

Since we reach 1:

true

For a number that enters a cycle:

... → A → B → C → A → B → C → ...

we will eventually find that A is already present in the HashSet.

Therefore, we know that the process is cycling and will never reach 1.

Java Solution

```
import java.util.*;
```

```
public class Main {
```

```
    // Calculates the sum of squares of the digits
```

```
    public static int sumOfSquares(int n) {
```

```
        int sum = 0;
```

```
        while (n > 0) {
```

```
int digit = n % 10;

sum += digit * digit;

n /= 10;

}

return sum;

}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();

    HashSet<Integer> seen = new HashSet<>();

    while (n != 1 && !seen.contains(n)) {

        seen.add(n);

        n = sumOfSquares(n);

    }

    System.out.println(n == 1);

}
```



Solution:

1. **Read the input:** `main()` reads the integer `n`, matching the Input Format.
2. **Create a HashSet:** Create a `HashSet<Integer>` named `seen` to store the numbers that have already occurred during the process. This helps us detect an infinite cycle.
3. **Calculate the sum of squared digits:** The `sumOfSquares()` method extracts each digit using `n % 10`, squares the digit, adds it to `sum`, and removes the last digit using `n / 10`.
4. **Repeat the process:** Continue calculating the sum of the squares of the digits while `n` is not 1 and the current value has not already been seen.
5. **Detect a cycle:** Before calculating the next value, add the current `n` to the `HashSet`. If the same value appears again, it means the process has entered a cycle and will never reach 1.
6. **Check the final result:** When the loop ends, there are two possibilities. If `n == 1`, the number is happy. Otherwise, a cycle was detected, so the number is not happy.
7. **Print the result:** Print `true` if `n == 1`; otherwise, print `false`, matching the Output Format.

Complexity Analysis

- **Time Complexity:** $O(\log n)$ for each transformation because we process every digit of the number. The process reaches either 1 or a cycle after a limited number of transformations.
- **Space Complexity:** $O(k)$, where `k` is the number of distinct values stored in the `HashSet` for cycle detection.